

DXC Step Tracker Web Application Redeployment Process

This document is a technical and comprehensive review of the process of migrating & redeployment of the [DXC Step Tracker Tool](#) for further usage & adoption throughout DXC.

The Architecture:

The overall architecture is a Python programme that uses the Streamlit library for frontend/web development. This is then in turn connected to several systems including the following:

- Microsoft Entra (Authentication)
- Microsoft OneDrive (trusted Internal Data storage)
- Supabase PostgreSQL Database (Data storage)
- Streamlit Cloud (Hosting Provider)
- GitHub (version control)

To successfully redeploy the application these aspects must be configured correctly.

The Process:

1. **Local Deployment** – Before anything the recommendation is to duplicate (clone) build and run the application locally to both test, debug and make any additional required changes to the applications codebase, this can be done from the [Github repository](#) and by following the instructions stated in README.md
2. **Streamlit Hosting** – If you have chosen to maintain hosting provider with streamlit cloud please navigate to the [community cloud dashboard](#) and create a new app. This new application should point at the newly cloned repository but will require a new unique app URL.
3. **Secrets File** – The application is now deployed, although will not functionally operate until the secrets file has been configured (App → Settings → Secrets). This process is also documented within the REAME.md but includes the following:

```
SUPABASE_URL = "" # supabase database url  
SUPABASE_KEY = "" # supabase api connectivity key
```

```
ADMIN_EMAILS = [ ] # a list of all user emails who should have admin access to the platform
```

```
client_id = "" # client ID of azure app registration
tenant_id = "" # directory or tenant ID
client_secret = "" # generated client secret (Certificates & Secrets)
```

Note: some of these fields will not be able to be populated until all steps are completed

- Microsoft App Registration** – To enable connectivity to microsoft and outlook we need to create a new [app registration](#). Once created we need to configure this registration for usage by generating a client secret (Certificates & Secrets), Under Authentication add in the appropriate redirect URL (URL chosen in streamlit cloud) and select the following in API graph permissions,

Microsoft Graph (4)				
email	Delegated	View users' email address	No	...
Files.ReadWrite.All	Delegated	Have full access to all files user can access	No	...
profile	Delegated	View users' basic profile	No	...
User.Read	Delegated	Sign in and read user profile	No	...

Once done so, populate the Streamlit secrets file with the relevant client-id, tenant-id & client-secret.

- Database Creation** – We now need to select a database to use, for the purpose of this use case we used [Supabase's free tier database](#). Although alternative PostgreSQL relational databases can be used. Begin by creating a new project, then copy the following schema into the new project/database. Following this add the appropriate supabase-url & supabase-key to the Streamlit secrets folder. Finally configure RLS properties to allow anon interactions with all 3 of the tables (Auth handled separately by Entra anyway).

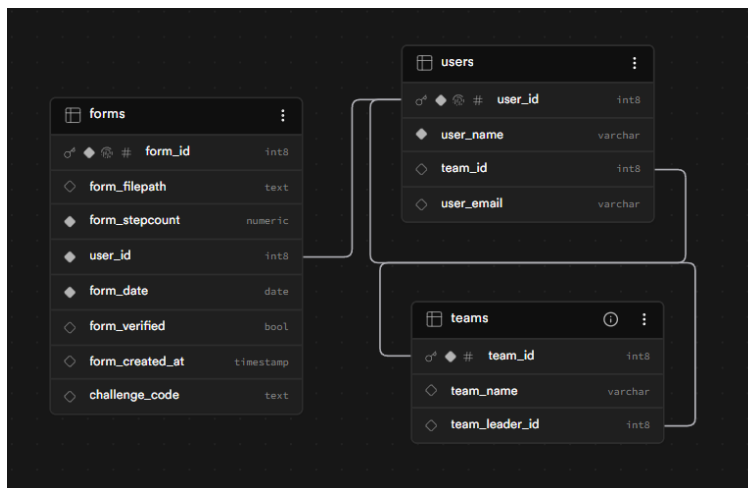


Table: forms

Name	Type	Constraints
form_id (pk)	int8	Primary Unique Identity
form_filepath	text	Nullable
form_stepcount	numeric	
user_id (fk)	int8	
form_date	date	
form_verified	bool	Nullable
form_created_at	timestamp	Nullable
challenge_code	text	Nullable

Table: teams

Name	Type	Constraints
team_id (pk)	int8	Primary Identity
team_name	varchar	Nullable
team_leader_id (fk)	int8	Nullable

Table: users

Name	Type	Constraints
user_id (pk)	int8	Primary Unique Identity
user_name	varchar	
team_id (fk)	int8	Nullable
user_email	varchar	Nullable

Additionally In the leaderboard page we use 2 Custom SQL query calls these can be found within the [README.md](#).

- `get_team_step_totals`
- `get_user_step_totals`

These have the purpose of quickly totaling and returning a view of both user and team totals in an optimized and scalable manner without having to do this within the application code – these aren't required although will help optimize the data retrieval on this page.

6. Other Implementation Considerations – The currently deployed version of the GitHub uses two separate automated workflows for:

- Keeping the Supabase DB active (ping-supabase.yml)
- Keeping the Streamlit hosted web app active (runner.py)

Within both there are variables that will need changing, importantly the URL of the streamlit application but also the endpoint to ping the supabase DB.

Additional Information:

For help or guidance upon this process please contact drew.wandless@dxs.com

Relevant documents that may aid this process include:

- [DXCStepTracker/README.md at main · DWandless/DXCStepTracker](#)
- <https://docs.streamlit.io/streamlit-community-cloud>
- <https://learn.microsoft.com/en-us/entra/identity-platform/quickstart-register-app>
- <https://supabase.com/docs/guides/auth/redirect-urls>
- [Create Your Own Supabase Database In 5 Minutes | Refine](#)
- [Use Microsoft Entra to authenticate users - Streamlit Docs](#)